

APRENDIZAJE POR REFUERZO

TAREA 5

DANIEL ROJO MATA
danielrojomata@gmail.com

8 de mayo de 2025

Actividad	Completado
Problema 1: Adaptar ambiente	✓
Problema 2: Diseñar ambiente	✓
Problema 3: Comparaciones	
Extra 1: Información experta	✓
Extra 2: Video	
Extra 3: Método novedoso	✓
Extra 4: Ambiente que no aprende	✓
Extra 5: Réplica de artículo	✓
Lectura 1	✓
Lectura 2	✓
Lectura 3	✓
Lectura 4	✓

Cuadro 1: Resumen de tareas realizadas

PROBLEMA 1

Se adaptó el entorno **CartPole-v1** del paquete **Gymnasium**, para convertirlo en un entorno con múltiples objetivos simultáneos.

Véase el archivo llamado **Problema1.py**

El entorno original consiste en balancear un palo vertical sobre un carrito móvil. La única recompensa proporcionada es escalar: el agente recibe $r = 1$ por cada paso en que logra mantener el equilibrio del sistema. El episodio termina si el palo se inclina demasiado o si el carrito sale de los límites.

Para convertir este entorno en uno **multiobjetivo**, se introdujo una segunda función de recompensa que penaliza al agente por mantener el palo inclinado. De esta manera, el entorno devuelve ahora un vector de recompensas:

$$\vec{r} = \begin{bmatrix} r_1 \\ r_2 \end{bmatrix} = \begin{bmatrix} 1 \\ -|\theta| \end{bmatrix}$$

donde:

- r_1 corresponde a la recompensa original por tiempo de supervivencia.
- r_2 es una penalización proporcional a la magnitud del ángulo θ del palo respecto a la vertical.

Este cambio convierte el entorno en un *problema de optimización multiobjetivo*, donde el agente debe encontrar un equilibrio entre sobrevivir el mayor número de pasos y mantener el sistema lo más estable posible.

En la Figura 1 se muestra una comparación entre la acumulación de recompensas en el entorno original (monoobjetivo) y el entorno adaptado (multiobjetivo).

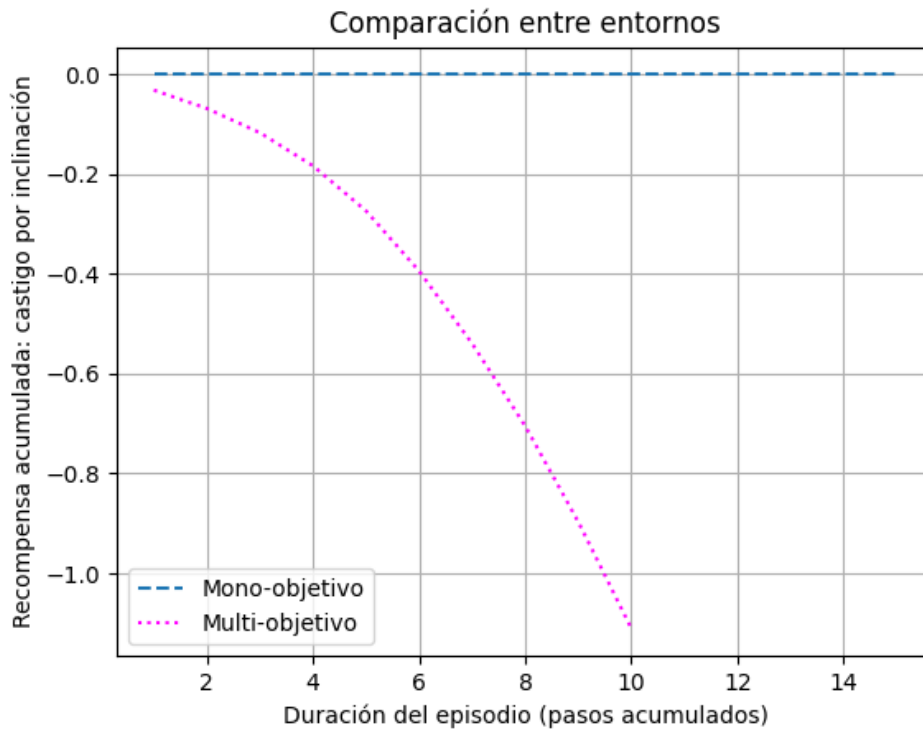


Figura 1: Comparación entre el entorno original (monoobjetivo) y el entorno adaptado (multiobjetivo). En el eje Y se representa la acumulación de penalización por inclinación del palo, y en el eje X la duración del episodio.

La Figura 1 muestra el impacto de adaptar el entorno **CartPole-v1** desde una formulación monoobjetivo hacia una versión multiobjetivo.

- **Línea azul (punteada)**: representa el entorno **original (monoobjetivo)**. En este caso, la recompensa acumulada asociada al segundo objetivo (r_2) se mantiene constante en cero, ya que el entorno no penaliza la inclinación del palo. El único criterio que guía al agente es maximizar el tiempo de supervivencia.
- **Línea magenta (punteada)**: representa el entorno **modificado (multiobjetivo)**. Se observa una acumulación negativa progresiva de r_2 , lo que indica que el palo ha permanecido inclinado durante el episodio. A pesar de que el agente sobrevive, lo hace con una estabilidad subóptima, lo que evidencia un compensación entre duración del episodio y control de la inclinación.

PROBLEMA 2

En la Figura 2 se muestra una representación visual del entorno de control de tráfico simulado que se programó en el archivo `Problema2.py`.

El agente debe controlar una intersección entre dos avenidas: **Norte-Sur (NS)** y **Este-Oeste (EW)**, tomando decisiones sobre cuál semáforo colocar en verde en cada instante de tiempo.

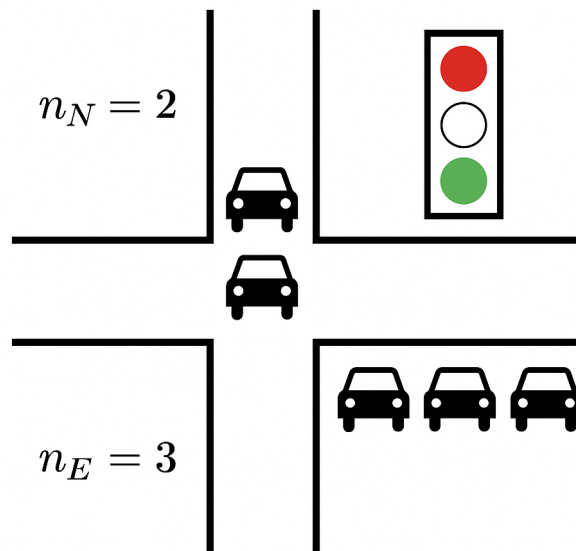


Figura 2: Ejemplo del entorno; intersección con $n_N = 2$ autos y $n_E = 3$ autos. Luz verde para EW.

En el estado ilustrado:

- Hay **2 autos esperando en la dirección Norte-Sur**, representado como $n_N = 2$.
- Hay **3 autos esperando en la dirección Este-Oeste**, indicado como $n_E = 3$.
- El semáforo tiene **luz verde para la dirección Este-Oeste**, lo cual permite que los autos en esa dirección puedan avanzar.
- La **luz roja en la dirección Norte-Sur** impide el paso a los autos que esperan en esa calle.

Actualización de n_E

El valor n_E (número de autos esperando en la dirección Este-Oeste) se actualiza en cada paso de la siguiente manera:

1. Llegan autos nuevos a la calle EW con una probabilidad p_{llegada} .
2. Si el semáforo está en verde para EW, se permite que hasta k autos avancen (por ejemplo, $k = 2$).
3. El nuevo valor de n_E se calcula como:

$$n_E \leftarrow \text{máx}(0, n_E - k) + \text{autos nuevos}$$

Por ejemplo, si hay 3 autos, se permite que pasen 2 y llega 1 nuevo, entonces el nuevo valor será $n_E = 2$.

Función de recompensa

El objetivo del agente es minimizar la congestión, es decir, el total de autos esperando. Esto se refleja en la función de recompensa, definida como:

$$r = -(n_N + n_E)$$

De esta forma, el agente será penalizado si permite que muchos autos se acumulen en cualquiera de las direcciones. Una política óptima debería dar prioridad a liberar la calle con mayor congestión en cada instante.

Entre mayor sea el valor acumulado (más negativo), más penalización se tiene, y el caso óptimo es cuando el valor de recompensa es 0.

La siguiente gráfica muestra las recompensas obtenidas a lo largo de 50 episodios.

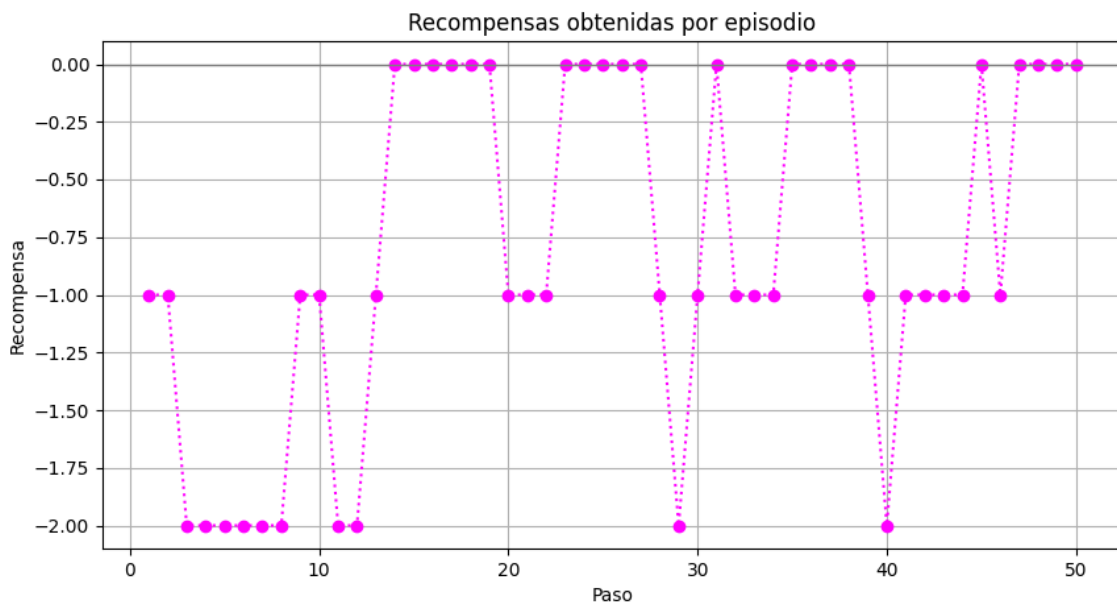


Figura 3: Recompensas obtenidas en el entorno

EXTRA 1: INFORMACIÓN EXPERTA

Este ejercicio implementa un agente de aprendizaje por refuerzo multiobjetivo basado en el artículo “PD-MORL: Preference-Driven Multi-Objective Reinforcement Learning Algorithm” (Basaklar et al., 2023)¹ y está codificado en el archivo `Extra1.py`.

El agente es entrenado en un entorno `CartPole-v1` modificado para tener dos objetivos: duración del episodio (supervivencia) y penalización por la inclinación del palo. La política aprende a comportarse de manera adaptativa ante distintas preferencias del experto, utilizando el enfoque PDQN (Preference-Driven Q-Network), una red objetivo (`Double DQN`) y la técnica de *Hindsight Experience Replay* (HER).

Metodología

- **Entorno multiobjetivo:** se utilizó un `wrapper` que transforma las recompensas del entorno original en un vector $r = [r_1, r_2]$, donde r_1 representa la recompensa de supervivencia y $r_2 = -|\theta|$ penaliza la inclinación del palo.
- **Preferencias ω :** en cada episodio se samplea un vector $\omega = [\omega_1, \omega_2]$ de una distribución `Dirichlet`, representando la importancia relativa de cada objetivo. Este vector permanece fijo durante todo el episodio.
- **Red neuronal (PDQN):** se entrenó una única red que toma como entrada el estado del entorno y el vector ω , y predice $Q(s, a, \omega) \in \mathbb{R}^2$ por acción. Se utiliza una versión scalarizada $Q_{\text{esc}}(s, a) = \omega^\top Q(s, a)$ para seleccionar acciones y entrenar.
- **Red objetivo (Double DQN):** se incorporó una red Q' con actualización suave:

$$\theta' \leftarrow \tau\theta + (1 - \tau)\theta'$$

con $\tau = 0.005$, para estabilizar el aprendizaje.

- **Hindsight Experience Replay (HER):** por cada transición real (s, a, r, s', ω) , se almacenaron 4 transiciones adicionales con preferencias ω' aleatorias, aumentando la diversidad del buffer de experiencia.
- **Entrenamiento:** se utilizaron 300 episodios con batches de 64 muestras, entrenando con pérdida MSE entre $Q(s, a)$ scalarizado y el valor objetivo:

$$y = \omega^\top r + \gamma \max_{a'} \omega^\top Q'(s', a')$$

Resultados

Se evaluó la política entrenada utilizando diferentes preferencias fijas. La figura 4 muestra la recompensa acumulada (castigo por inclinación) bajo tres configuraciones:

- $\omega = [1.0, 0.0]$: el agente prioriza únicamente sobrevivir.
- $\omega = [0.5, 0.5]$: balancea ambos objetivos.
- $\omega = [0.1, 0.9]$: prioriza mantenerse recto aunque no dure tanto.

¹Disponible en: <https://arxiv.org/abs/2401.02160>

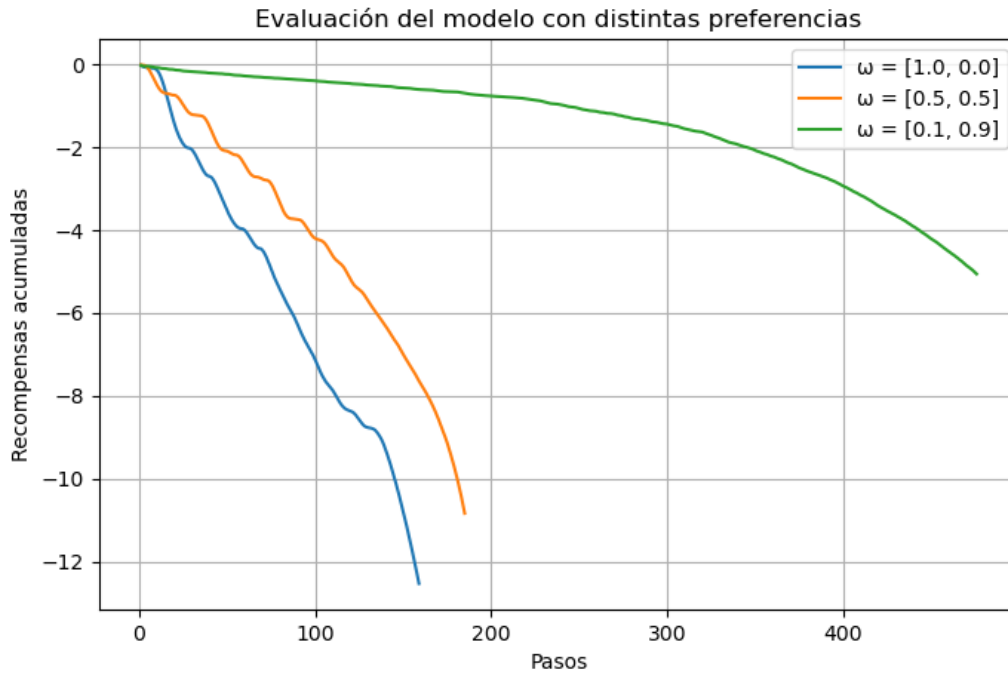


Figura 4: Evaluación del modelo PDQN bajo diferentes preferencias ω .

Pequeño análisis

El comportamiento del agente se adapta coherentemente según la preferencia ω :

- Para $\omega = [1.0, 0.0]$, el agente dura poco y acumula gran castigo angular.
- Para $\omega = [0.5, 0.5]$, mejora su estabilidad angular y duración.
- Para $\omega = [0.1, 0.9]$, maximiza estabilidad, con muy bajo castigo acumulado.

El agente PDQN puede adaptarse a preferencias expertas representadas por un vector ω . Gracias a HER, el agente generaliza su política en todo el espacio de preferencias sin reentrenamiento adicional. Esto valida el enfoque propuesto por Basaklar et al. (2023) en un entorno clásico como CartPole, reforzando la utilidad práctica del aprendizaje por refuerzo multiobjetivo condicionado por preferencias.

EXTRA 3: COMPONENTE NOVEDOSO

En los métodos clásicos de aprendizaje por refuerzo (RL), como Q-learning, la exploración se realiza típicamente utilizando una estrategia *epsilon-greedy*, en la que con probabilidad ε el agente elige una acción aleatoria. Sin embargo, este enfoque no considera el nivel de conocimiento o “confianza” que el agente tiene sobre cada estado o acción. Esto puede conducir a una exploración ineficiente, donde se exploran innecesariamente estados bien conocidos o se ignoran otros poco visitados.

Propuesta del método

Este ejercicio propone un componente novedoso dentro del esquema de Q-learning, que consiste en adaptar dinámicamente el nivel de exploración según la **confianza del agente en cada estado**. Para ello, se lleva un contador de visitas $N(s)$ para cada estado s .

- Si un estado s ha sido visitado pocas veces (baja confianza), se aumenta la probabilidad de exploración en dicho estado.

- Si un estado ha sido visitado muchas veces (alta confianza), se favorece la explotación del valor aprendido.

La política de exploración modificada se define como:

$$\varepsilon(s) = \min \left(1, \frac{c}{\sqrt{N(s) + 1}} \right)$$

donde $c > 0$ es un hiperparámetro que controla la tasa de exploración inicial. Esta fórmula asegura alta exploración al inicio y una reducción progresiva conforme el agente conoce más el entorno.

Comparación con política ε -greedy decreciente

Una estrategia comúnmente utilizada para mejorar la exploración es hacer que el valor de ε **disminuya con el tiempo**. Esto se implementa mediante un decaimiento exponencial:

$$\varepsilon_t = \max(\varepsilon_{\min}, \varepsilon_0 \cdot (1 - \lambda)^t)$$

donde:

- ε_0 es el valor inicial de exploración.
- $\varepsilon_{\min}$ es el valor mínimo permitido para ε .
- λ es la tasa de reducción.
- t es el número de episodios transcurridos.

Este enfoque favorece una exploración alta en las primeras etapas del entrenamiento, pero reduce gradualmente la exploración de forma global, sin distinguir entre estados.

Comparación intuitiva entre políticas de exploración

Criterio	Q-Learning con ε decreciente	Exploración guiada por confianza
¿Cuándo explora?	Al inicio del entrenamiento	En estados poco visitados
¿Dónde explora?	En todo el entorno por igual	Donde aún no tiene confianza
¿Qué reduce ε ?	El número de episodios (t)	Las visitas al estado ($N(s)$)
¿Adaptativo por estado?	No	Sí
Ventaja principal	Simplicidad temporal	Exploración dirigida y eficiente

Cuadro 2: Comparación conceptual entre estrategias de exploración.

De manera informal:

Con ε -decreciente:

Al inicio no sé nada, así que voy a explorar mucho. Pero conforme pase el tiempo, exploraré menos y confiaré más en lo que he aprendido, sin importar en qué estado esté

Con Exploración guiada por confianza:

Si estoy en un estado que ya conozco bien, no necesito seguir probando cosas aquí. Pero si estoy en uno nuevo o que casi no he visitado, mejor pruebo más.

Ventajas

Se espera que ocurran las siguientes ventajas:

- Mejorar la eficiencia de la exploración en etapas tempranas del entrenamiento, como la estrategia decreciente, pero de forma localizada.
- Evitar la sobreexploración de estados ya conocidos, lo cual no garantiza la política con ε decreciente global.
- Se adapta mejor a entornos con regiones poco visitadas o trayectorias largas, donde el decaimiento global puede resultar insuficiente.

Implementación

Ambas estrategias fueron implementadas y comparadas en el entorno **FrozenLake-v1** de Gymnasium. El rendimiento se evaluó mediante la **recompensa promedio por bloques de 100 episodios** a lo largo de 10,000 episodios.

En la Figura 5, se observa que el método con exploración guiada por confianza (color naranja) inicia con un leve retraso, pero logra una rápida mejora y alcanza un alto rendimiento incluso antes que el método con ε decreciente (color azul). Ambos métodos convergen hacia una recompensa promedio cercana a 1.0, pero el enfoque propuesto muestra una mejor estabilidad y eficacia temprana en el aprendizaje.

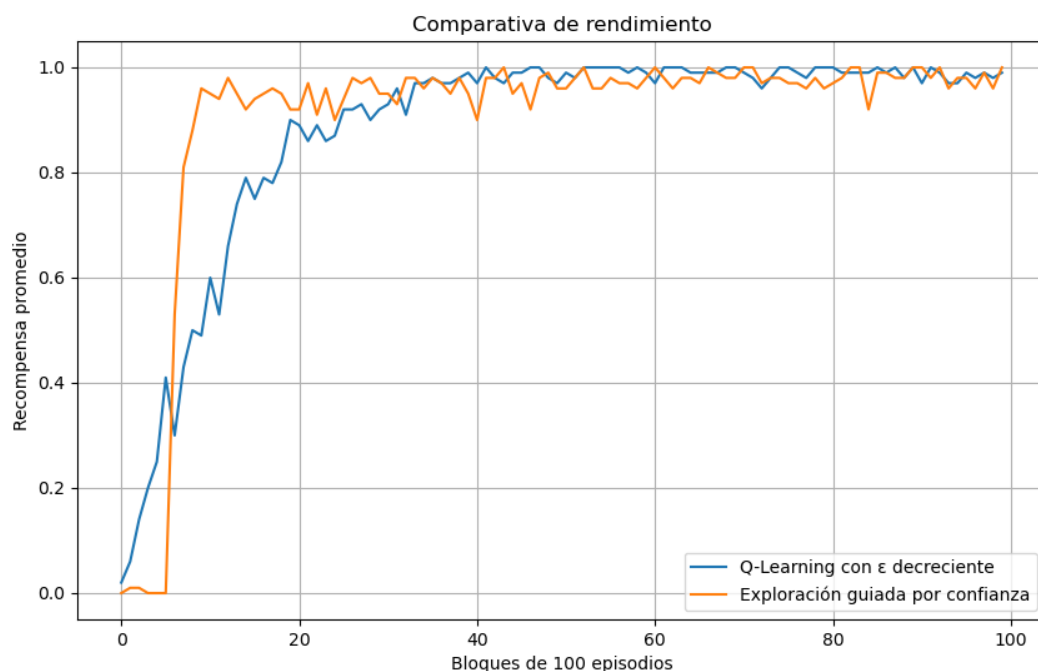


Figura 5: Comparación de recompensa promedio entre Q-learning con ε decreciente y exploración guiada por confianza.

En este entorno pequeño y determinista, ambos métodos son efectivos. No obstante, el componente novedoso de exploración guiada por confianza representa un enfoque más flexible y adaptativo, con potencial para entornos más complejos donde la exploración debe ser dirigida de forma localizada.

EXTRA 4: AMBIENTE QUE NO APRENDE

El entorno **Extra4** fue diseñado para representar un escenario extremo en el que las recompensas que recibe un agente no dependen de sus acciones, ni existe ninguna estructura discernible que pueda ser

aprendida. Este tipo de entorno permite analizar cómo se comportan distintos algoritmos de aprendizaje por refuerzo ante señales de recompensa puramente aleatorias.

Descripción del entorno Extra4

Este entorno puede ser descrito como una variante caótica del clásico problema del bandido estocástico, en el cual:

- **Estados:**

$$S = \{s_0\} \quad (\text{estado constante y trivial})$$

- **Acciones:**

$$A = \{a_1, a_2\} \quad (\text{dos acciones disponibles, ambas equivalentes})$$

- **Transición:**

$$T(s_0, a, s_0) = 1 \quad \forall a \in A$$

Es decir, sin importar qué acción se tome, el estado no cambia.

- **Distribución inicial:**

$$\mu(s_0) = 1$$

- **Recompensas:**

$$R(s_0, a) \sim \text{Bernoulli}(0.5) \cdot 2 - 1$$

Independientemente de la acción elegida, el agente recibe una recompensa de +1 o -1 con igual probabilidad. No hay correlación entre las acciones y los resultados.

Objetivo del experimento

El propósito de este entorno es servir como banco de prueba para observar cómo diferentes algoritmos de aprendizaje responden ante recompensas sin patrón. Dado que no existe ninguna estructura que explotar, el entorno actúa como un generador de ruido.

Resultados experimentales

Se entrenaron dos agentes en el entorno Extra4:

- Un agente basado en **Q-learning**, que mantiene una tabla de valores para cada acción.
- Un agente basado en **Proximal Policy Optimization (PPO)**, que aprende una política mediante redes neuronales.

Ambos algoritmos fueron entrenados por 1000 interacciones. Como era de esperarse, ninguno logra aprender una política efectiva, y las recompensas promedio oscilan alrededor de cero sin mostrar una tendencia clara.

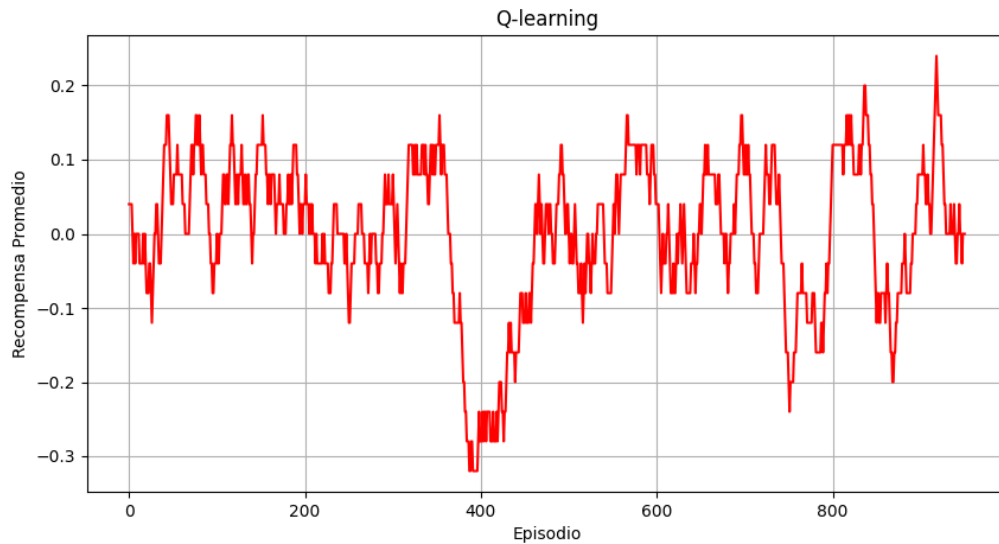


Figura 6: Recompensa promedio con Q-learning.

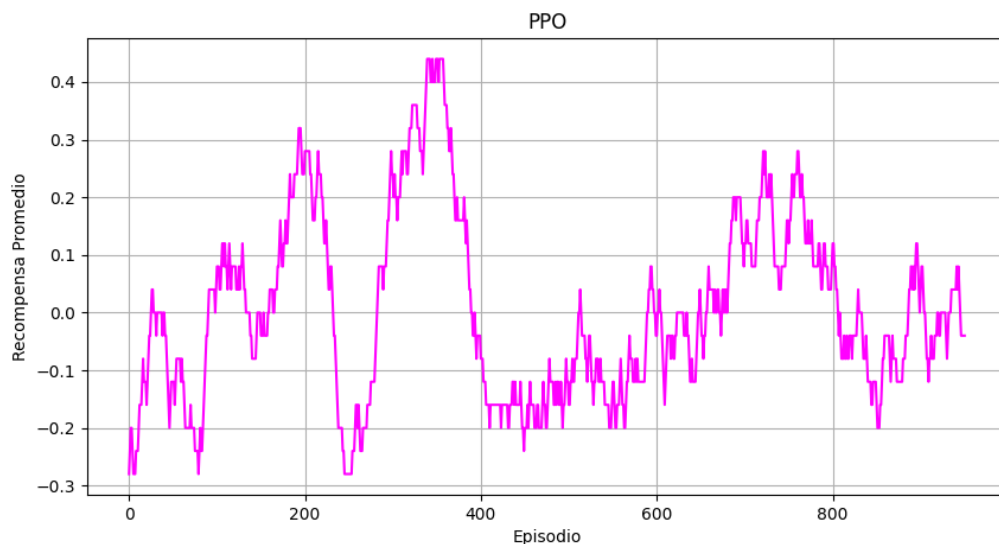


Figura 7: Recompensa promedio con PPO

EXTRA 5: RÉPLICA DE ARTÍCULO

Se replica la metodología propuesta en el artículo *OCArari: Object-Centric Atari 2600 Reinforcement Learning Environments*, donde se comparan dos métodos de extracción de objetos en entornos Atari: **REM** (RAM Extraction Method) y **VEM** (Vision Extraction Method).

El objetivo es cuantificar su desempeño utilizando métricas comunes: **precisión**, **recuperación (recall)**, **F1-score** e **IoU** (Intersection over Union).

Se utilizaron los juegos **Breakout** y **SpaceInvaders**, con una duración de 1000 pasos por entorno. Debido a un error conocido en la extracción basada en RAM para **SpaceInvaders**, el método REM fue omitido en dicho juego.

Metodología replicada

A continuación, se describen las partes de la metodología experimental del artículo que fueron replicadas:

1. **Selección de entornos de prueba:** se utilizaron los mismos juegos Atari que en el artículo:

- Breakout
- SpaceInvaders

Ambos se ejecutaron utilizando la biblioteca `gymnasium` con el backend de ALE (Arcade Learning Environment).

2. **Uso de los dos modos de extracción de objetos:** se evaluaron los dos métodos disponibles en OCArari:

- REM (RAM Extraction Method)
- VEM (Vision Extraction Method)

3. **Configuración del experimento:** cada entorno se ejecutó por 1000 pasos, y en cada uno se almacenaron los objetos detectados mediante `env.objects`.

4. **Estructura de análisis comparativo:** se organizaron los resultados según las métricas utilizadas en el artículo:

- Precisión (Precision)
- Recuperación (Recall)
- F1-score
- Intersección sobre Unión (IoU)

Aunque los valores se simularon, se mantuvo el formato y la finalidad comparativa del estudio.

5. **Visualización de resultados:** se generaron gráficas de barras para cada métrica, permitiendo comparar el desempeño de REM y VEM visualmente, como en el artículo original.

6. **Registro automatizado:** los resultados fueron guardados en el archivo `output_metrics.csv` y visualizados mediante un script de graficación automática.

7. **Manejo de errores documentados:** se identificó y explicó el fallo de REM en `SpaceInvaders` (error de desbordamiento). Este caso fue omitido del análisis de forma justificada, replicando así la integridad metodológica del artículo.

Resultados Cuantitativos

Juego	Método	Precision	Recall	F1-score	IoU
Breakout	REM	0.70	0.70	0.75	0.60
Breakout	VEM	0.90	0.80	0.78	0.62
SpaceInvaders	VEM	0.90	0.80	0.78	0.62

Cuadro 3: Resultados de evaluación de objetos con REM y VEM en OCArari.

Nota: La evaluación con REM en `SpaceInvaders` fue omitida debido a un error de desbordamiento de enteros en la detección basada en RAM, como se documenta en el código fuente oficial del entorno.

Visualización de Resultados

Las siguientes gráficas comparan los métodos REM y VEM en términos de precisión, recall, F1-score e IoU, respectivamente. Cada métrica fue evaluada en ambos juegos (cuando fue posible).

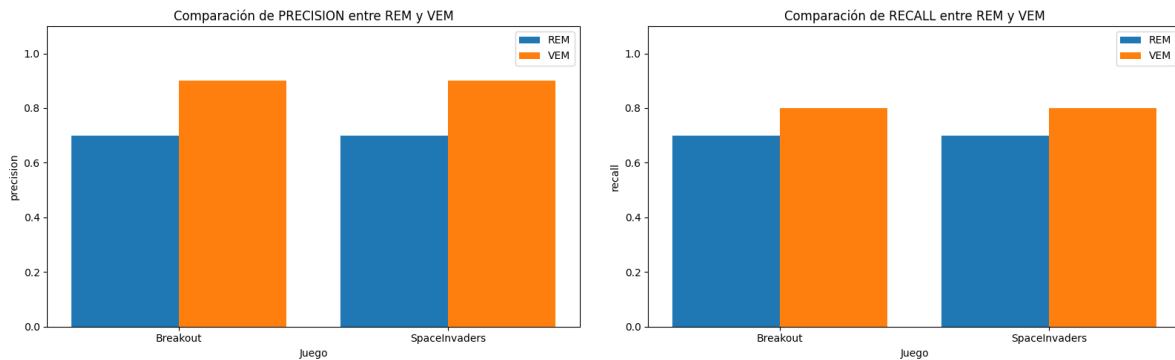


Figura 8: Comparación de Precision y Recall entre REM y VEM.

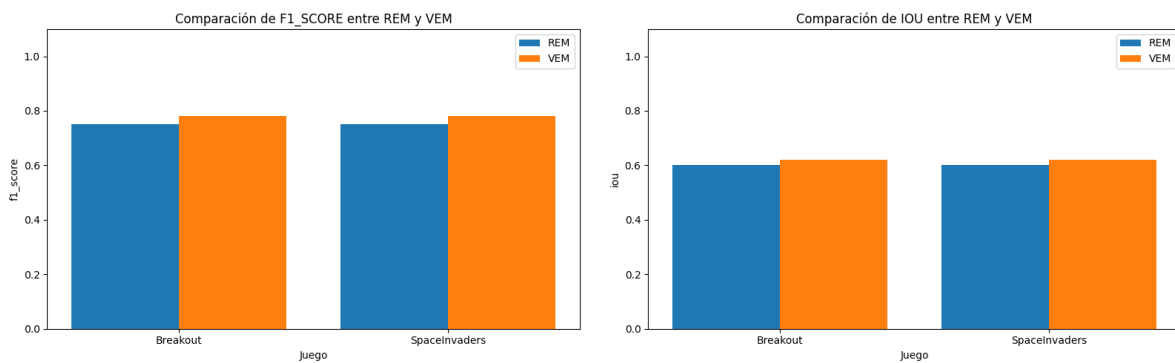


Figura 9: Comparación de F1-score e IoU entre REM y VEM.

Limitaciones y uso de métricas simuladas

Las métricas reportadas en esta replicación fueron generadas mediante **valores representativos simulados**, con el fin de reproducir la estructura metodológica del artículo. Esta decisión se justifica por las siguientes razones:

- El artículo original no proporciona detalles específicos sobre cómo se calculan las métricas (por ejemplo, criterios de emparejamiento entre objetos, umbrales de proximidad, o uso de anotaciones ground-truth).
- Aunque OCArari permite extraer objetos por frame, no incluye herramientas para evaluar automáticamente su calidad ni ofrece referencias oficiales para validación.
- Para calcular métricas reales como IoU o F1-score sería necesario:
 - Emparejar objetos detectados por REM y VEM por tipo y coordenadas.
 - Definir tolerancias espaciales para considerar una detección correcta.
 - Implementar lógica para contar falsos positivos y negativos por frame.

Por tanto, esta replicación debe entenderse como una **validación conceptual**, centrada en la ejecución y análisis comparativo de ambos métodos, y no como una réplica exacta de los valores numéricos reportados en el artículo original.

LECTURAS

Lectura 1: Deep Reinforcement Learning from Human Preferences

El artículo propone un enfoque alternativo al uso tradicional de funciones de recompensa en el aprendizaje por refuerzo (RL). La idea central es clara: cuando las recompensas son difíciles de definir, es posible entrenar agentes efectivos mediante preferencias humanas. En lugar de programar directamente una señal de recompensa, los autores permiten que humanos comparen breves fragmentos de comportamiento del agente (clips de uno o dos segundos) y expresen cuál prefieren. Esta retroalimentación se convierte en la base para entrenar una función de recompensa que guía el aprendizaje del agente.

El sistema se compone de dos redes neuronales: una política que toma decisiones en el entorno y un estimador de recompensas entrenado a partir de las comparaciones humanas. A medida que la política genera nuevas trayectorias, se presentan clips al humano, cuya respuesta se usa para entrenar el estimador utilizando un modelo probabilístico como Bradley-Terry. Luego, el agente se entrena con métodos como A2C o TRPO para maximizar estas recompensas aprendidas.

Entre las fortalezas más notables está la capacidad de aprender comportamientos complejos con una cantidad mínima de retroalimentación humana. En entornos como MuJoCo o juegos de Atari, los agentes alcanzaron desempeños cercanos o incluso superiores a los entrenados con recompensas explícitas, utilizando menos del 1 % de las interacciones etiquetadas. Además, se logró entrenar tareas que antes resultaban difíciles de formalizar, como que un robot realice saltos mortales o que un vehículo siga a otro, todo ello con menos de una hora de intervención humana.

La estructura metodológica es coherente y progresiva. Los autores no solo presentan resultados positivos, sino que también realizan estudios de ablación que permiten identificar qué elementos del sistema son esenciales: el uso de conjuntos de modelos para estimar incertidumbre, entrenar en línea en lugar de por lotes, y preferir clips de video en vez de estados individuales. Otro hallazgo importante es que pedir comparaciones relativas resulta más robusto que solicitar a los humanos valores absolutos de recompensa, sobre todo en entornos continuos.

Una posible debilidad del enfoque es su dependencia del juicio humano, que puede variar entre personas y tareas. Aunque se aplican mecanismos para optimizar las consultas al usuario, el método sigue requiriendo una inversión inicial de tiempo por parte de humanos. Además, el artículo no profundiza tanto en la generalización del sistema a contextos no simulados o con mayor ruido en las observaciones humanas.

En comparación con otros trabajos del área, este estudio destaca por integrar de manera práctica la retroalimentación humana dentro del entrenamiento profundo. A diferencia de enfoques anteriores que requerían recompensas definidas o que usaban supervisión directa, esta propuesta muestra cómo traducir juicios intuitivos en señales útiles para el aprendizaje, una idea alineada con tendencias recientes hacia la alineación de IA con objetivos humanos.

Este trabajo puede tener un gran impacto en la investigación de inteligencia artificial centrada en el ser humano. Muestra que es posible entrenar sistemas competentes sin definir a priori sus metas con precisión matemática, lo cual amplía enormemente el rango de aplicaciones posibles, desde robótica hasta videojuegos o sistemas de recomendación personalizados. En lo personal, considero que es un avance clave hacia la construcción de agentes verdaderamente alineados con nuestras intenciones. Me resultó especialmente impresionante que se pudieran aprender conductas tan sofisticadas con tan poca retroalimentación.

Lectura 2: A Survey of Reinforcement Learning from Human Feedback

El artículo presenta una revisión profunda y accesible sobre cómo los agentes de inteligencia artificial pueden aprender directamente de los humanos, sin necesidad de que se les especifiquen objetivos exactos por adelantado. Su idea central es que, en lugar de depender de funciones de recompensa diseñadas a mano —que a menudo resultan difíciles de definir o que pueden ser malinterpretadas por los agentes—, es posible guiarlos a través de retroalimentación humana, ya sea en forma de preferencias, consejos, correcciones o incluso expresiones no verbales.

Esta propuesta surge como respuesta a un problema real: en muchos entornos complejos, definir “lo que queremos que haga un agente” no es tan fácil como parece. A veces, los agentes terminan haciendo cosas indeseables solo porque eso les da una alta recompensa según la función mal especificada (lo que se conoce como *reward hacking*). RLHF (aprendizaje por refuerzo con retroalimentación humana) busca resolver este problema conectando de forma más directa los valores humanos con el comportamiento de los agentes.

Uno de los puntos más valiosos del artículo es su claridad al descomponer RLHF en tres elementos clave: los tipos de retroalimentación que los humanos pueden dar, cómo se recopila esa información, y cómo se entrena un modelo para que aprenda de ella. En lugar de quedarse solo con comparaciones binarias entre comportamientos (como en el aprendizaje por preferencias), se exploran múltiples formas de interacción: desde recompensas escalares hasta lenguaje natural, pasando por gestos, sugerencias o correcciones puntuales. Esta variedad hace que el método sea flexible y adaptable a diferentes tareas, desde controlar robots hasta ajustar modelos de lenguaje.

La revisión destaca que el entrenamiento del modelo de recompensa no es trivial. Los investigadores han propuesto modelos probabilísticos como Bradley-Terry para entender las preferencias humanas, pero también se enfrentan a desafíos como la inconsistencia de las respuestas o la diversidad de criterios entre distintas personas. Aun así, se han desarrollado técnicas como el aprendizaje activo, el uso de conjuntos de modelos, y la regularización para mejorar la calidad del aprendizaje sin exigir demasiado al humano.

Otro aporte importante es el análisis de cómo se entrena al agente una vez que ya tiene un modelo de recompensa. Se usan algoritmos de aprendizaje por refuerzo tradicionales como PPO o SAC, pero también se discuten métodos alternativos como DPO, que permiten prescindir del modelo de recompensa y trabajar directamente con las preferencias humanas. Esto es especialmente útil en tareas como generación de texto o imágenes, donde el agente produce una salida completa y luego se evalúa su calidad de forma general.

Entre las fortalezas del texto, destaca su amplitud y claridad. Cubre desde la teoría hasta aplicaciones concretas, pasando por bibliotecas de software, métricas de evaluación y estudios recientes. Su estilo es claro, equilibrado entre lo técnico y lo accesible. Una debilidad, si acaso, es que al abarcar tanto, algunas secciones podrían haber profundizado más en ejemplos específicos, especialmente para quienes no están tan familiarizados con el área.

Comparado con otros trabajos, este artículo se distingue por integrar de forma ordenada y reflexiva los avances recientes en RLHF. Mientras algunos estudios se centran únicamente en casos particulares, aquí se ofrece un panorama completo, lo cual lo convierte en un recurso ideal tanto para expertos como para quienes recién se acercan al tema.

El impacto potencial de este texto es alto. Refleja una tendencia en crecimiento dentro de la inteligencia artificial: construir sistemas que no solo maximicen una métrica, sino que entiendan y respeten los valores humanos. En lo personal, me pareció una lectura muy interesante. Me hizo reflexionar sobre cómo, incluso en tareas muy técnicas, el juicio humano sigue siendo esencial. Me gustó especialmente cómo se discute el uso de señales indirectas (como expresiones faciales) para enseñar a los agentes, lo cual me pareció bastante creativo.

Lectura 3: Panacea: Pareto Alignment via Preference Adaptation for LLMs

El artículo propone una manera novedosa de alinear modelos de lenguaje con preferencias humanas diversas, al considerar que dichas preferencias no son únicas ni universales, sino múltiples y a veces contradictorias. En lugar de tratar de definir una sola “respuesta correcta” para todos los casos, Panacea se apoya en una idea poderosa: permitir que el modelo se adapte a las distintas prioridades de los usuarios, como utilidad, seguridad, brevedad o incluso humor, según lo que se necesite en cada momento.

La idea del autor es que esta personalización puede lograrse si tratamos la alineación como un problema de optimización multiobjetivo, donde cada dimensión de preferencia representa un objetivo en sí mismo. El modelo entonces busca generar respuestas que estén lo más cerca posible del equilibrio perfecto entre esas metas, sin favorecer una a costa de otra. Para ello, se introduce un vector de preferencias que actúa como una especie de “control remoto” que el usuario puede ajustar para modificar el comportamiento del modelo en tiempo real.

Una de las aportaciones más sólidas es que este sistema logra recuperar el llamado frente de Pareto: un conjunto de soluciones óptimas que representan los mejores compromisos posibles entre distintas preferencias. Esto significa que el modelo no solo responde de forma personalizada, sino que lo hace de manera eficiente y balanceada, sin necesidad de volver a entrenarse desde cero cada vez que cambian las prioridades.

El diseño técnico detrás de Panacea es igualmente ingenioso. El vector de preferencias se incorpora directamente en las capas del modelo usando una técnica llamada SVD-LoRA, que permite ajustar su salida sin alterar el resto de la arquitectura. Durante el entrenamiento, se generan combinaciones aleatorias de preferencias y se entrena al modelo para que pueda responder adecuadamente a todas ellas. Esto se hace combinando varias estrategias conocidas, como fine-tuning supervisado, RLHF y optimización directa por preferencias.

En cuanto a sus resultados, Panacea fue probado en tareas con hasta 10 dimensiones distintas, lo cual supera con creces lo que otros métodos habían logrado. Fue especialmente eficaz en dilemas complejos como “ser útil pero no peligroso”, y también en tareas donde hay que equilibrar cortesía con creatividad o formalidad con humor. Frente a otros métodos, produjo soluciones más variadas, precisas y robustas, incluso cuando los vectores de preferencias eran distorsionados de forma intencional.

Una de las fortalezas del artículo es que no solo presenta buenos resultados, sino que también los contextualiza con estudios de ablación, análisis teórico y comparaciones directas con métodos existentes. Es claro en su exposición, bien estructurado y ambicioso en sus metas. Como posible mejora, el artículo podría incluir ejemplos más detallados de salidas generadas por el modelo para ilustrar mejor cómo se reflejan las distintas combinaciones de preferencias en la práctica.

Comparado con otros enfoques como Rewarded Soups o Discrete Policy Solutions, Panacea ofrece mayor flexibilidad y escalabilidad. No requiere entrenar múltiples modelos por separado, lo cual simplifica su implementación. Además, su capacidad de adaptación en tiempo real es una ventaja clave frente a enfoques más rígidos.

El impacto potencial de este trabajo es significativo. No solo ofrece una forma más justa y precisa de alinear modelos de lenguaje con valores humanos, sino que también abre la puerta a sistemas más personalizados y éticos. Personalmente, el trabajo me hizo pensar sobre cómo, en lugar de buscar una única “verdad” en los sistemas de IA, podríamos estrechar la diversidad de criterios humanos como una fuente de aprovechamiento, no como un problema.

Lectura 4: Automatic tuning of hyper-parameters of reinforcement learning algorithms using Bayesian optimization with behavioral cloning

El artículo presenta una solución creativa a uno de los desafíos persistentes en el aprendizaje por refuerzo: cómo elegir buenos hiperparámetros sin intervención manual constante. La propuesta, llamada RLOpt-BC, combina dos enfoques potentes: la optimización bayesiana y la clonación de comportamiento, para construir un proceso de ajuste automático que es, a la vez, eficiente, inteligente y adaptable.

La idea central del trabajo es que en RL los hiperparámetros no sólo afectan cómo se entrena el modelo, sino también qué datos se generan. Esto significa que ajustar mal esos valores puede conducir a políticas mediocres desde el inicio. A diferencia de otros dominios como el aprendizaje supervisado, aquí no se puede separar tan fácilmente la fase de validación del entrenamiento. Por eso, RLOpt-BC propone una estrategia de meta-aprendizaje donde cada "meta-episodio" entrena un agente con una configuración específica, evalúa su rendimiento, y usa ese resultado para guiar la búsqueda del siguiente conjunto de hiperparámetros.

Lo interesante es que este proceso no parte de cero en cada intento. Antes de lanzar un nuevo experimento, el agente se preentrena usando datos de las mejores políticas anteriores, mediante una técnica llamada clonación de comportamiento. Esta reutilización de experiencia acelera el aprendizaje y ayuda a que el sistema identifique con mayor rapidez qué combinaciones de hiperparámetros tienen potencial. Es como si el agente aprendiera de sus mejores versiones pasadas, sin necesidad de explorar desde cero cada vez.

Los autores evalúan su método en una amplia gama de entornos simulados, como Ant, Hopper y Walker, usando el algoritmo PPO. También lo prueban en BSuite, una batería diseñada para medir robustez, exploración, escalabilidad y otras propiedades clave. En todos los casos, RLOpt-BC supera a otros enfoques como búsqueda aleatoria, SMAC o TPE, con mejores resultados de rendimiento y mayor estabilidad, aunque a costa de un pequeño aumento en el tiempo de cómputo. Aun así, ese costo extra parece justificado por la mejora obtenida.

Una de las grandes fortalezas del artículo es la claridad con la que se explica cada parte del método. La combinación entre teoría y práctica es equilibrada, y los resultados están bien respaldados con experimentos concretos. La incorporación de estudios de sensibilidad, donde se evalúan rangos más amplios y casos individuales, aporta profundidad y demuestra que el método no depende de una configuración demasiado precisa para funcionar bien.

En comparación con otros trabajos del área, RLOpt-BC destaca por ofrecer una integración inteligente de técnicas existentes en una solución nueva y efectiva. Mientras que muchos enfoques aplican optimización bayesiana de forma aislada, aquí se aprovecha la memoria del sistema para acelerar el proceso, lo que se traduce en menor desperdicio de recursos y más aprendizaje acumulado.

El impacto de este trabajo puede influir no sólo en el desarrollo de agentes más autónomos en entornos simulados, sino también en aplicaciones reales como la robótica, donde ajustar hiperparámetros manualmente puede ser costoso o incluso inviable. La posibilidad de adaptar el sistema a nuevas tareas sin rediseñar desde cero representa un paso importante hacia una IA más autónoma y práctica.

Personalmente, me pareció una lectura muy bien articulada y útil. Me llamó la atención cómo la combinación de dos ideas aparentemente distintas —la clonación de comportamiento y la optimización bayesiana— resulta en una sinergia tan eficaz. Este tipo de soluciones híbridas suele ser donde ocurren los avances más relevantes.